

Updating the Score as Aliens Are Shot Down

To write a live score onscreen, we update the value of `stats.score` whenever an alien is hit, and then call `prep_score()` to update the score image. But first, let's determine how many points a player gets each time they shoot down an alien:

```
settings.py    def initialize_dynamic_settings(self):
               --snip--

               # Scoring settings
               self.alien_points = 50
```

We'll increase each alien's point value as the game progresses. To make sure this point value is reset each time a new game starts, we set the value in `initialize_dynamic_settings()`.

Let's update the score in `_check_bullet_alien_collisions()` each time an alien is shot down:

```
alien_invasion.py def _check_bullet_alien_collisions(self):
                  """Respond to bullet-alien collisions."""
                  # Remove any bullets and aliens that have collided.
                  collisions = pygame.sprite.groupcollide(
                      self.bullets, self.aliens, True, True)

                  if collisions:
                      self.stats.score += self.settings.alien_points
                      self.sb.prep_score()
                  --snip--
```

When a bullet hits an alien, Pygame returns a collisions dictionary. We check whether the dictionary exists, and if it does, the alien's value is added to the score. We then call `prep_score()` to create a new image for the updated score.

Now when you play *Alien Invasion*, you should be able to rack up points!

Resetting the Score

Right now, we're only prepping a new score *after* an alien has been hit, which works for most of the game. But when we start a new game, we'll still see our score from the old game until the first alien is hit.

We can fix this by prepping the score when starting a new game:

```
alien_invasion.py def _check_play_button(self, mouse_pos):
                  --snip--
                  if button_clicked and not self.game_active:
                      --snip--
                      # Reset the game statistics.
                      self.stats.reset_stats()
                      self.sb.prep_score()
                      --snip--
```
